

Conciertos

Se parte del siguiente conjunto de tablas que modelan la gestión de entradas de conciertos:

- **Concierto** (idconcierto (PK), nombre, ciudad, fecha, tickets, precio, idgrupo (FK-Grupo))
- **Grupo** (idgrupo (PK), nombre, estilo, activo)
- **Cliente** (nif (PK), nombre, apellidos, direccion, cp, ciudad)
- **Compra** (idcompra (PK), idconcierto(FK->Concierto), nif (FK->Cliente), n_tickets)

En el material se proporciona el script SQL, con TODAS las restricciones adicionales y una carga de datos inicial, que se recomienda revisar con detalle. Sobre dicho modelo de tablas, se pide implementar las siguientes transacciones:

```
1. public void comprar (Date fecha, String nif, int grupo,
    int tickets) throws PersistenceException;
```

Este método inserta una nueva compra en la tabla correspondiente y resta el número de entradas compradas de la tabla concierto. Si no hay suficientes entradas disponibles en el momento de la compra, lanzará una excepción.

```
2. public void desactivar (int grupo) throws
    PersistenceException;
```

Este método desactiva el grupo cuyo id aparece en el campo de entrada de la transacción de la tabla Grupos (modifica el campo activo a 0). Esto quiere decir que el grupo no va a realizar ninguna gira y no está disponible.

Elimina todos los registros asociados de todas las tablas (salvo el registro modificado en la tabla Grupo). No se puede utilizar la opción cascade.

```
3. public List 1 <Grupo> consultarGrupos() throws
    PersistenceException;
```

Este método consulta todos los grupos y su información relativa, recupera toda la información vinculada a los grupos, concierto, clientes y compras, para su visualización posterior.

Para su resolución es obligatorio **utilizar un grafo de entidades** que recupere toda la información solicitada (grupos, conciertos, compras y clientes).

Excepciones y pruebas automáticas

Los códigos de las excepciones y codificaciones de error, se proporcionan, en `IncidentException` ² e `IncidentError` respectivamente, en el paquete `es.ubu.lsi.service.conciertos`.

¹ El uso de `List` en *Hibernate* puede dar problemas en el uso de grafos de entidades. Una alternativa es cambiar en el modelo de entidades al tipo `Set` para las colecciones a varios, en lugar de `List`. El tipo de datos que devuelve siempre va a ser `List` independientemente de lo empleado en el modelo de clases.

² Se han añadido más constructores que los utilizados en ejercicios de prácticas por si fueran necesarios.

Queda a juicio del alumnado generar las correspondientes excepciones con su correspondiente código de error en cada transacción. Si se considera que falta algún error en la enumeración, se pueden añadir a los valores del tipo enumerado nuevos errores y códigos de excepción. **No se pueden eliminar los valores ya definidos.**

Se os proporciona una batería de pruebas mínima con *tests* de ejemplo sobre las transacciones solicitadas. El conjunto de pruebas garantiza un mínimo funcionamiento, pero deben tenerse en cuenta e implementarse todos los posibles casos de error adicionales que se puedan dar en las transacciones (datos de entrada incorrectos, lógica de negocio incorrecta, etc.)

Esas pruebas están implementadas utilizando el servicio JPA y en algún caso se puede combinar con código JDBC (utilizando un pool de conexiones con JNDI), tal y como se ha venido trabajando en la asignatura previamente en la parte de JDBC.

Las pruebas "automáticas" adicionales se programarán en la clase `es.ubu.lsi.test.TestClient`. Dicha clase contiene un método `main` para ejecutar los *tests* y unos *tests* básicos sobre las transacciones implementadas.

Si se implementa algún test adicional (en otra clase `TestClientAlumno`), en que los datos de entrada pudieran generar varios errores, se deja a decisión del alumnado cuál de los errores se recoge y lanza en primer lugar, no prefijando ningún orden particular.

La salida esperada con los tests ya proporcionados en `TestClient`, debería ser similar a la que aparece a continuación aunque depende de cómo se haya configurado el método `ToString`:

```
Iniciando...
Probando el servicio...
Framework y servicio iniciado...
Insertar compra correcta
    OK compra bien insertada
    OK actualiza bien los tickets del concierto
Insertar compra con grupo incorrecto
    OK detecta correctamente que NO existe el grupo
Insertar compra con cliente incorrecto
    OK detecta correctamente que NO existe el cliente
Insertar compra con concierto incorrecto
    OK detecta correctamente que NO existe el concierto
Insertar compra con concierto incorrecto
    OK detecta correctamente que NO existen estos tickets disponibles
Información completa con grafos de entidades...
Grupo [idgrupo=1, nombre=Los Burning, estilo=Rock, activo=1]
    Concierto [idconcierto=1, nombre=Rock Tardio, ciudad=Madrid, fecha=2023-11-01 21:00:00.0, tickets=97, precio=25.5]
        Compra [idcompra=3, cliente=Cliente [nif=7666832Y, nombre=Luis, apellidos=Rodriguez Garcia, direccionPostal=DireccionPostal [direccion=C/Atlantico 12, codigoPostal=3333, ciudad=Burgos]], concierto=Concierto [idconcierto=1, nombre=Rock Tardio, ciudad=Madrid, fecha=2023-11-01 21:00:00.0, tickets=97, precio=25.5], tickets=3]
            Cliente [nif=7666832Y, nombre=Luis, apellidos=Rodriguez Garcia, direccionPostal=DireccionPostal [direccion=C/Atlantico 12, codigoPostal=3333, ciudad=Burgos]]
        Compra [idcompra=4, cliente=Cliente [nif=7352353T, nombre=Luis, apellidos=Rodriguez Garcia, direccionPostal=DireccionPostal [direccion=C/Hueso Roto 15, codigoPostal=9999, ciudad=Palencia]], concierto=Concierto [idconcierto=1, nombre=Rock Tardio, ciudad=Madrid, fecha=2023-11-01 21:00:00.0, tickets=97, precio=25.5], tickets=3]
            Cliente [nif=7352353T, nombre=Luis, apellidos=Rodriguez Garcia, direccionPostal=DireccionPostal [direccion=C/Hueso Roto 15, codigoPostal=9999, ciudad=Palencia]]
        Compra [idcompra=5, cliente=Cliente [nif=1111111F, nombre=Luis, apellidos=Rodriguez Garcia, direccionPostal=DireccionPostal [direccion=C/Mayor 5, codigoPostal=9999, ciudad=Palencia]], concierto=Concierto [idconcierto=1, nombre=Rock Tardio, ciudad=Madrid, fecha=2023-11-01 21:00:00.0, tickets=97, precio=25.5], tickets=10]
            Cliente [nif=1111111F, nombre=Luis, apellidos=Rodriguez Garcia, direccionPostal=DireccionPostal [direccion=C/Mayor 5, codigoPostal=9999, ciudad=Palencia]]
        Compra [idcompra=6, cliente=Cliente [nif=1111111F, nombre=Luis, apellidos=Rodriguez Garcia, direccionPostal=DireccionPostal [direccion=C/Mayor 5, codigoPostal=9999, ciudad=Palencia]], concierto=Concierto [idconcierto=1, nombre=Rock Tardio, ciudad=Madrid, fecha=2023-11-01 21:00:00.0, tickets=97, precio=25.5], tickets=3]
            Cliente [nif=1111111F, nombre=Luis, apellidos=Rodriguez Garcia, direccionPostal=DireccionPostal [direccion=C/Mayor 5, codigoPostal=9999, ciudad=Palencia]]
Grupo [idgrupo=2, nombre=Laura Mancini, estilo=Romantico, activo=1]
    Concierto [idconcierto=3, nombre=Gira Balada, ciudad=Madrid, fecha=2023-11-15 22:30:00.0, tickets=100, precio=27.5]
        Compra [idcompra=2, cliente=Cliente [nif=7352353T, nombre=Luis, apellidos=Rodriguez Garcia, direccionPostal=DireccionPostal [direccion=C/Hueso Roto 15, codigoPostal=9999, ciudad=Palencia]], concierto=Concierto [idconcierto=3, nombre=Gira Balada, ciudad=Madrid, fecha=2023-11-15 22:30:00.0, tickets=100, precio=27.5], tickets=10]
```

```

        Cliente [nif=7352353T, nombre=Luis, apellidos=Rodriguez Garcia, direccionPostal=DireccionPostal [direccion=C/Hueso
Roto 15, codigoPostal=9999, ciudad=Palencia]]
        Compra [idcompra=1, cliente=Cliente [nif=1111111F, nombre=Luis, apellidos=Rodriguez Garcia,
direccionPostal=DireccionPostal [direccion=C/Mayor 5, codigoPostal=9999, ciudad=Palencia]], concierto=Concierto [idconcierto=2,
nombre=Gira Balada, ciudad=Madrid, fecha=2023-11-01 22:00:00.0, tickets=100, precio=35.0], tickets=5]
        Cliente [nif=1111111F, nombre=Luis, apellidos=Rodriguez Garcia, direccionPostal=DireccionPostal [direccion=C/Mayor 5,
codigoPostal=9999, ciudad=Palencia]]
OK Sin excepciones en la consulta completa y acceso posterior
Desactivar grupo correcto
        OK grupo correctamente desactivado Desactivar
grupo incorrecto
        OK detecta correctamente que NO existe el grupo
FIN.....

```

Estructura del Trabajo

Para la realización de esta entrega se seguirá el mismo diseño ya utilizado en las dos últimas sesiones prácticas. Por lo tanto, se seguirá utilizando el patrón de diseño DAO junto con JPA. En el material adjunto en el aula virtual se proporciona el esqueleto del proyecto JPA a completar por el alumnado.

En `es.ubu.lsi.service` está la interfaz de servicio `Service` que debe ser implementada por el alumnado, conteniendo los métodos correspondientes a las transacciones solicitadas.

Es **obligatorio utilizar las bibliotecas de usuario** de sesiones anteriores de prácticas con JPA (proyecto `user_lib_JPA` y bibliotecas de usuario asociadas). En concreto son necesarias, cuatro bibliotecas de usuario: *FileSystemContext*, *Hibernate*, *Oracle* y *SLF4J*.

Se proporciona el script SQL en el subdirectorio `./sql/` del proyecto con nombre `script.sql`. Dicho script **NO debe ser modificado** puesto que se utiliza en la ejecución de las pruebas automáticas.

Código a implementar y orden sugerido de implementación

- 1- Paquete `es.ubu.lsi.model.conciertos` con las clases/entidades persistentes de JPA necesarias para el modelo de datos.
 - Se valorará el implementar tipos embebidos adicionales si se da el caso.
 - Para evitar errores con el uso de grafos de entidades y la ejecución de los *tests*, se recomienda colecciones de tipo `Set` en lugar de tipo `List` para las relaciones a varios.
- 2- Paquete `es.ubu.lsi.dao.conciertos`: con los DAO necesarios que heredan de `JpaDAO`.
 - Se recuerda que en todos estos DAO se debe implementar el método abstracto `findAll` con la típica consulta que devuelve TODAS las instancias de la entidad correspondiente.
- 3- Paquete `es.ubu.lsi.service.conciertos`: implementación del servicio concreto `ServiceImpl` (ojo: **extends** `PersistenceService` **implements** `Service`) dando código concreto al cuerpo de los métodos (transacciones) de la interface `Service`.
- 4- Paquete `es.ubu.lsi.service.test`: implementación de la clase `TestClient`, con los

tests automáticos, sobre la implementación de las transacciones con JPA. Se proporciona ya el código de esta clase. Este código tendrá errores en la primera apertura del proyecto ya que faltarían las clases a implementar de los puntos anteriores.

- 5- En caso de que quieran ampliarse con *tests* adicionales, se copiará el código original y se ampliará en una clase `TestClientAlumno` en ese mismo paquete.

Condiciones de realización y entrega

- La práctica se realizará en grupos de tres personas, los mismos del resto de entregas de la asignatura.
- En caso de alguna irregularidad podría solicitarse una defensa del trabajo en horario de tutorías, a fijar con los profesores responsables de la asignatura. Una exagerada diferencia entre el resultado de este trabajo y el del examen del tema 4, suponen razón suficiente para solicitar esa defensa. En caso de no ser capaz de defender el trabajo, la calificación será modificada acorde a la defensa realizada.
- Cada integrante del grupo debe entregar en una única carpeta comprimida (ZIP) con todos los ficheros necesarios en el buzón de entrega correspondiente en el aula virtual y contener una sola carpeta que usando la función de Eclipse "Open Projects from File System" importe el proyecto y se pueda ejecutar.
- El nombre del fichero entregado seguirá este formato, sin tildes: `Nombre_1erApellido_Nombre_1erApellido_Nombre_1erApellido.zip`
- Comprobad que la solución está bien integrada en el esquema de carpetas que se os facilita ya que no se corregirá ni puntuará una práctica que no se importe y ejecute correctamente.
- También hay que incluir en la carpeta un archivo PDF con los nombres completos de los participantes en la práctica.
- No se admitirán entregas por correo electrónico, así que no apuréis la fecha de entrega para evitar problemas técnicos a la hora de entregar y recordad que hay que dar al botón enviar cuando esté subido el fichero definitivo para que no se quede en "Borrador".

Comentarios adicionales

Además, es **muy importante** tomar en consideración los siguientes comentarios para la resolución del proyecto:

- La unidad de persistencia JPA debe llamarse "**Conciertos**".
- En este ejercicio los DAO deben incluir métodos adicionales. Es decisión del alumnado qué clases DAO son necesarias y qué código colocar en cada uno.
- Se considera que TODAS las asociaciones en el modelo de clases son **bidireccionales**.
- En UBUVirtual se proporciona el proyecto con la solución parcial a completar por los alumnos. Se recuerda que debe ajustarse correctamente el *classpath* con las bibliotecas de usuario necesarias.